

Nombre Manuel Fernando Guzmán Arias ID 4948560
Carrera Ingeniería de Software y Minería de Datos Fecha 2026-10-08
Repositorio: <https://github.com/FerGuzman65/pruebas-de-software-a-lab-6-exepciones-Lab6>

LABORATORIO # 6 – MANEJO DE ERRORES

Introducción

En este laboratorio, los estudiantes aplicarán manejo de excepciones y pruebas unitarias con `pytest` sobre la clase `ExceptionsDemo` desarrollada en Python.

Los objetivos incluyen:

- Detectar y manejar errores usando `try-except`.
- Validar métodos individuales ante entradas inválidas.
- Verificar interacción de funciones y manejo de errores.
- Probar el sistema completo asegurando que las excepciones se manejen correctamente.

Objetivo

Que los alumnos comprendan cómo probar y manejar excepciones en funciones de Python, detectando errores y garantizando que el programa no falle abruptamente.

Descripción del programa

La clase `ExceptionsDemo` incluye métodos que pueden lanzar o manejar las siguientes excepciones:

Excepción	Significado
<code>ZeroDivisionError</code>	Error por dividir entre cero
<code>TypeError</code>	Error por tipo de dato incorrecto
<code>IndexError</code>	Índice fuera de rango en listas
<code>KeyError</code>	Clave inexistente en diccionarios

Excepción	Significado
FileNotFoundException	Archivo no encontrado al abrirlo
IOException	Error de lectura/escritura de archivo
AttributeError	Intentar acceder a un atributo que no existe
Excepción personalizada MyException	Error definido por el programador para reglas propias

Los métodos permiten:

- División de números (divide)
- Acceso a listas (access_list)
- Acceso a diccionarios (access_dict)
- Lectura de archivos (read_file)
- Acceso a atributos (access_attribute)
- Validación de números positivos (check_positive)

Actividades

Instrucciones generales

Las siguientes actividades se llevarán a cabo **de manera sincronizada con el profesor**, quien explicará el procedimiento y las ventajas de cada paso:

1. Ingresa al enlace de **Git** y **realice un fork del repositorio**.
2. Abre el proyecto y revisa las funciones disponibles en los archivos proporcionados
3. Realiza las actividades descritas y explicadas por el docente.
4. Entrega un breve análisis de los errores encontrados y una conclusión sobre la practica.
5. Realiza un commit y push en git agregando los cambios al final de la practica.

Actividad 1: Excepciones

1. Abra el archivo test_exceptions.py.
2. Ejecute en la terminal el comando pytest.
3. Verifique que todas las pruebas se ejecuten correctamente.
4. Preste atención a la explicación del docente sobre la ejecución de pruebas y el manejo de excepciones.
5. Agregue o modifique funciones para capturar y manejar correctamente las excepciones identificadas en la Actividad 1.

Actividad 2: Excepciones

Modifica cada función de la clase ExceptionsDemo del archivo Exceptions.py para incorporar bloques try-except de manera que, ante la ocurrencia de cualquier excepción, el programa no se detenga y el flujo del main continúe ejecutándose de forma controlada.

¿Que excepciones se agregaron a cada función ?

Se agregaron bloques try-except en los seis métodos de la clase ExceptionsDemo. Cada método captura la excepción que puede ocurrir, imprime un mensaje de error y permite que el programa siga ejecutándose en lugar de detenerse:

Método	Excepción que se agregó
divide(a, b)	ZeroDivisionError, TypeError
access_list(lst, index)	IndexError
access_dict(dic, key)	KeyError
read_file(filename)	FileNotFoundError, IOError
access_attribute(obj)	AttributeError
check_positive(number)	MyException (personalizada)

Actividad 3: Documentación

Entrega de este reporte de laboratorio

Agregar sección de anexos de capturas de pantalla de Exceptions.py y de test_exceptions.py.

Análisis de los errores encontrados

Al ejecutar pytest por primera vez salieron dos errores en las pruebas. El primero fue usar `pytest.raises` como una llamada suelta y sin indicarle la excepción esperada, lo que provocaba `ValueError: You must specify at least one parameter to match on`; se corrigió usándolo como context manager con `with` y pasándole la excepción. El segundo fue llamar a `access_attribute` con dos argumentos cuando el método solo recibe uno, lo que daba `TypeError: access_attribute() takes 2 positional arguments but 3 were given`; se corrigió quitando el argumento extra. Con esos cambios las siete pruebas detectaron correctamente `ZeroDivisionError`, `TypeError`, `IndexError`, `KeyError`, `FileNotFoundError`, `AttributeError` y `MyException`.

Después de agregar los bloques `try-except` de la Actividad 2, esas mismas siete pruebas pasaron a fallar con el mensaje `Failed: DID NOT RAISE`. Esto pasa porque cuando el método captura la excepción dentro de sí mismo, la excepción ya no sale del método, y `pytest.raises` justamente espera que salga. En la salida de cada prueba se ve que el mensaje de error controlado sí se imprime, así que el manejo de excepciones funciona como lo pide la Actividad 2; lo que queda desactualizado son las pruebas, que tendrían que revisar el mensaje impreso en vez de esperar la excepción.

Conclusiones

En este laboratorio entendí la diferencia entre dejar que una excepción se propague y atraparla con `try-except`. Dejarla propagarse sirve para las pruebas, porque así `pytest.raises` puede comprobar que el código detecta el error; atraparla con `try-except` es lo que conviene en un programa en uso, para que un error no detenga toda la ejecución. También aprendí que `pytest.raises` se usa con `with` y necesita la excepción esperada, que un error en la propia prueba (como pasar más argumentos de los que acepta el método) hace que falle antes de llegar a revisar la excepción, y que al cambiar el manejo de errores de una función también hay que revisar las pruebas que dependían de su comportamiento anterior. Por último, repasé el flujo de Git: `fork`, `cambios`, `commit` y `push`.

Anexos

Anexo 1. `Exceptions.py` con los bloques `try-except`.

Exceptions.py

```
1  class MyException(Exception):
2      pass
3
4  class ExceptionsDemo:
5
6      def divide(self, a, b):
7          try:
8              result = a / b
9              print("Division result:", result)
10         except ZeroDivisionError:
11             print("Error: Division by zero is not allowed.")
12         except TypeError:
13             print("Error: Invalid type for division. Please provide numbers.")
14
15
16
17     def access_list(self, lst, index):
18         try:
19             print("Element:", lst[index])
20         except IndexError:
21             print("Error: Index is out of bounds.")
22
23
24     def access_dict(self, dic, key):
25         try:
26             print("Value:", dic[key])
27         except KeyError:
28             print("Error: Key does not exist in the dictionary.")
29
30     def read_file(self, filename):
31         try:
32             with open(filename, "r") as f:
33                 content = f.read()
34                 print(content)
35         except FileNotFoundError:
36             print("Error: File not found.")
37         except IOError:
38             print("Error: Could not read the file.")
39
40     def access_attribute(self, obj):
41         try:
42             print(obj.tnonexistent_attribute)
43         except AttributeError:
44             print("Error: The object does not have that attribute.")
45
46     def check_positive(self, number):
47         try:
48             if number < 0:
49                 raise MyException("The number must be positive.")
50             else:
51                 print("Valid number:", number)
52         except MyException as e:
53             print(f"Error: {e}")
```

Anexo 2. test_exceptions.py con las pruebas de pytest.

```
test_exceptions.py

1  import pytest
2  from Exceptions import ExceptionsDemo, MyException
3
4
5
6  def test_division_cero():
7      demo = ExceptionsDemo()
8      with pytest.raises(ZeroDivisionError):
9          demo.divide(10, 0)
10
11
12
13  def test_division_with_string ():
14      demo=ExceptionsDemo()
15      with pytest.raises(TypeError):
16          demo.divide(10, "string")
17
18
19  def test_access_list_out():
20      demo=ExceptionsDemo()
21      with pytest.raises(IndexError):
22          demo.access_list([1,3,4], 7)
23
24  def test_access_dic():
25      demo=ExceptionsDemo()
26      with pytest.raises(KeyError):
27          demo.access_dict({"color": "red", "forma": "cuadrada"}, "sabor")
28
29
30  def test_read_file():
31      demo=ExceptionsDemo()
32      with pytest.raises(FileNotFoundError):
33          demo.read_file("texts.txt")
34
35  def test_access_attribute():
36      demo=ExceptionsDemo()
37      with pytest.raises(AttributeError):
38          demo.access_attribute([1,3,4])
39
40
41  def test_check_positive():
42      demo=ExceptionsDemo()
43      with pytest.raises(MyException):
44          demo.check_positive(-5)
```

Anexo 3. Ejecución de pytest en la terminal.

```

pytest -v (Lab 6)

1  ===== test session starts =====
2  platform win32 -- Python 3.9.13, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\manue\AppData\Local\Programs\Python\Python39\python.exe
3  cachedir: .pytest_cache
4  rootdir: C:\Users\manue\OneDrive - Universidad Autonoma de Guadalajara\UAG\Pruebas de software\Parcial 2\Lab 6
5  collecting ... collected 7 items
6
7  test_exceptions.py::test_division_cero FAILED [ 14%]
8  test_exceptions.py::test_division_with_string FAILED [ 28%]
9  test_exceptions.py::test_access_list_out FAILED [ 42%]
10 test_exceptions.py::test_access_dic FAILED [ 57%]
11 test_exceptions.py::test_read_file FAILED [ 71%]
12 test_exceptions.py::test_access_attribute FAILED [ 85%]
13 test_exceptions.py::test_check_positive FAILED [100%]
14
15 ===== FAILURES =====
16 _____ test_division_cero _____
17
18     def test_division_cero():
19         demo = ExceptionsDemo()
20         with pytest.raises(ZeroDivisionError):
21             > demo.divide(10, 0)
22     E       Failed: DID NOT RAISE <class 'ZeroDivisionError'>
23
24 test_exceptions.py:9: Failed
25 ----- Captured stdout call -----
26 Error: Division by zero is not allowed.
27 _____ test_division_with_string _____
28
29     def test_division_with_string ():
30         demo=ExceptionsDemo()
31         with pytest.raises(TypeError):
32             > demo.divide(10, "string")
33     E       Failed: DID NOT RAISE <class 'TypeError'>
34
35 test_exceptions.py:16: Failed
36 ----- Captured stdout call -----
37 Error: Invalid type for division. Please provide numbers.
38 _____ test_access_list_out _____
39
40     def test_access_list_out():
41         demo=ExceptionsDemo()
42         with pytest.raises(IndexError):
43             > demo.access_list([1,3,4], 7)
44     E       Failed: DID NOT RAISE <class 'IndexError'>
45
46 test_exceptions.py:22: Failed
47 ----- Captured stdout call -----
48 Error: Index is out of bounds.
49 _____ test_access_dic _____
50
51     def test_access_dic():
52         demo=ExceptionsDemo()
53         with pytest.raises(KeyError):
54             > demo.access_dict({"color": "red", "forma": "cuadrada"}, "sabor")
55     E       Failed: DID NOT RAISE <class 'KeyError'>
56
57 test_exceptions.py:27: Failed
58 ----- Captured stdout call -----
59 Error: Key does not exist in the dictionary.
60 _____ test_read_file _____
61
62     def test_read_file():
63         demo=ExceptionsDemo()
64         with pytest.raises(FileNotFoundError):
65             > demo.read_file("texts.txt")
66     E       Failed: DID NOT RAISE <class 'FileNotFoundError'>
67
68 test_exceptions.py:33: Failed
69 ----- Captured stdout call -----
70 Error: File not found.
71 _____ test_access_attribute _____
72
73     def test_access_attribute():
74         demo=ExceptionsDemo()
75         with pytest.raises(AttributeError):
76             > demo.access_attribute([1,3,4])
77     E       Failed: DID NOT RAISE <class 'AttributeError'>
78
79 test_exceptions.py:38: Failed
80 ----- Captured stdout call -----
81 Error: The object does not have that attribute.
82 _____ test_check_positive _____
83
84     def test_check_positive():
85         demo=ExceptionsDemo()
86         with pytest.raises(MyException):
87             > demo.check_positive(-5)

```